

ALE 課程 9：期末案例一——MIRO 觸發 Paper Company 自動記 AR

Lecture

這是 ALE 課程的第一個期末案例，把 ale04 ~ ale08 學到的所有技巧串成一條真實的企業流程：

集團內公司 A (採購方) 對關聯公司 Paper Company (供應商，同時是集團內另一家公司) 做了 **Invoice Verification (MIRO)**，公司 A 認列應付帳款 (AP)。Paper Company 的帳上要自動鏡射一筆應收帳款 (AR) ——借記「公司 A 這位客戶」、貸記收入科目。

ale01 情境二問過這個問題，ale05 的三種觸發機制表也預告過答案：**MIRO 沒有原生的 Output Determination**，觸發時機必須自己用 **Enhancement** 掛勾。這題完整實作，而且因為是從零自訂（跟 ale10 的「站在標準流程上加值」形成對照），Message Type、Segment、Outbound、Inbound 全部自己來。

整體架構

```
[公司 A : MIRO 存檔]
├─ BAdI INVOICE_UPDATE ~ CHANGE_IN_UPDATE      ← 觸發時機 ( GUI : SE19 建
Implementation )
├─ ZCL_ALE09_MIRO_OUT=>enqueue                  ← 通過「安全閘」才寫一筆待送記錄到
ZALE09_QUEUE
                                                    ( 在 Update Task 內，跟 MIRO 同一
個 LUW 一起 commit )
[排程或手動：ZR_ALE09_SENDQ]
├─ ZCL_ALE09_MIRO_OUT=>send_pending              ← 掃待送清單→組 ZALE09
IDoc→MASTER_IDOC_DISTRIBUTE
├─ IDoc 走 ale02 的管線送出 ( 自我迴圈 )
[Paper Company : Inbound]
├─ Process Code ZALE09 → Z_IDOC_INPUT_ZALE09 ( 轉接器 )
├─ ZCL_ALE09_AR_POST=>process                    ← 驗證對應設定
→BAPI_ACC_DOCUMENT_CHECK / POST 記 AR
```

設計決策一：為什麼不直接在 BAdI 裡送 IDoc？——自己做一個 mini-NAST

ale05 講過手動 / Enhancement 觸發的表格裡，「待處理清單表」欄位是「無」，思考題還問過「即時呼叫失敗了誰會知道要重試」。這題的答案是**自己補上那個中間層**：

- **BAdI 掛勾點不能 COMMIT WORK** (en06 的 MESSAGE_TYPE_X Dump 教訓)，但 MASTER_IDOC_DISTRIBUTE 要 commit 才會真的送出。所以 BAdI 只做一件事：寫一筆「待送」記錄到 ZALE09_QUEUE。
- 選用 CHANGE_IN_UPDATE 方法 (2026-09-21 實測 INVOICE_UPDATE 的介面有 CHANGE_BEFORE_UPDATE / CHANGE_AT_SAVE / CHANGE_IN_UPDATE 三個時機；CHANGE_IN_UPDATE 在 Update Task 內執行，收到的是最終確定的發票號碼，寫入的待送記錄跟 MIRO 本身同一個 LUW 一起 commit——MIRO 失敗回滾，待送記錄也一起消失，不會有「發票不存在卻有待送記錄」的不一致)。

- 另有排程報表 **ZR_ALE09_SENDQ** 去掃 **ZALE09_QUEUE** 裡還沒送的記錄、組 IDoc、送出、標記已送。失敗的記錄會留在佇列裡等下一輪重試——這就是 **NAST** 加 **RSNAST00**、Change Pointer 加 **BD21** 的同一個架構，只是這次是你自己的。

設計決策二：安全閘——只有明確登記的組合才會觸發

MIRO 是每天在跑的真實現交易，掛上自訂邏輯會影響所有公司代碼的所有發票。沿用 en04 / en06 的作法：**ZALE09_CTRL** 控制表登記「來源公司代碼 + 供應商」的組合，沒登記的發票一律完全不動作。這張表同時扮演對應設定：這個供應商對應到哪個目標公司代碼、目標公司帳上的客戶編號、收入科目、文件類型，以及一個 **test_mode** 旗標。

設計決策三：Inbound 預設只「檢查」不「過帳」

Inbound 用 **BAPI_ACC_DOCUMENT_POST** 在目標公司代碼記一張真實的會計憑證。共用系統上這是不可輕易回頭的動作，所以 **test_mode** 預設打開時只呼叫 **BAPI_ACC_DOCUMENT_CHECK**（完整跑一遍所有檢查、但不寫入任何資料），IDoc 一樣走到 53，訊息註明「檢查通過、未過帳」。確認整條管線都正確之後，才由你明確關掉 **test_mode** 做真正過帳（並且要在測試專用的公司代碼進行）。

記帳分錄與 BAPI 的正負號慣例

Paper Company 認列 AR，分錄是：

科目	借 / 貸	金額 (amt_doccur)
客戶 (公司 A)	借	總額 (正數)
收入科目 (每個明細一行)	貸	各明細金額 (負數)

BAPI_ACC_DOCUMENT_POST 的規則是借方正、貸方負，整張憑證加總必須為零。借方走 **ACCOUNTRECEIVABLE**（客戶行）、貸方走 **ACCOUNTGL**（總帳行），三張表的 **itemno_acc** 用同一組項次串起來，金額都放在 **CURRENCYAMOUNT**（2026-09-21 實測 **BAPI_ACC_DOCUMENT_POST / _CHECK** 的介面與結構欄位）。

設計決策四：Inbound 冪等

發送方可能重送、**BD87** 可能重跑。Inbound 過帳前先用業務鍵檢查：把「來源發票號碼 + 會計年度」（14 碼）放進憑證表頭的 **ref_doc_no**，過帳前查 **BKPF** 同公司代碼、同 **xblnr** 是否已經有憑證——有就視為成功（回報「已存在」），避免重複記帳。

本題要新建的 ALE 物件（全部 GUI，比照 ale06 ~ ale08 的步驟）

- **Segment** (**WE31**，Field name 一律與 Data element 同名)：**Z1ALE09H** = **BUKRS** / **RE_BELNR** / **GJAHR** / **LIFRE** / **WAERS** / **BLDAT** / **BUDAT** / **XBLNR1**；**Z1ALE09I** = **RBLGP** / **EBELN** / **EBELP** / **WRBTR**。金額欄位用 Data Element **WRBTR**（2026-09-21 實測標準 Segment 對 **WRBTR** 的匯出長度是 15，足以容納 13 位數加小數）
- **Message Type** **ZALE09** (**WE81**)、**Basic Type** **ZALE09_BT01** (**WE30**：**Z1ALE09H** 1..1 → 底下 **Z1ALE09I** 1..999)、**WE82** 指派、全部 Set Release

學習目標

- 能為「沒有原生輸出框架」的交易資料設計完整的 Outbound 流程：掛勾點→安全閘→待送佇列→排程送出
- 能說明為什麼掛勾點只寫佇列、不直接送，並選對 BAdI 方法 (`CHANGE_IN_UPDATE`)
- 能設計並實作 Inbound：對應設定驗證、BAPI 檢查 / 過帳、冪等檢查、`test_mode` 保護
- 能正確處理 FI BAPI 的借貸正負號與 `itemno_acc` 串接
- 能獨立完成一條自訂訊息類型的端對端流程 (Outbound + Inbound + 所有 Partner Profile / Distribution Model 設定)

事前準備

- ale06 ~ ale08 完成，特別是 `WE57` / `BD51` / `WE42` / `WE20` 的操作已經熟練
- 找出測試用的主資料組合 (這個系統的資料因人而異，請自行確認)：來源公司代碼 (公司 A)、供應商 (代表 Paper Company，在公司 A 有維護)、目標公司代碼、目標公司帳上的客戶 (代表公司 A) 與收入科目。強烈建議目標公司代碼使用測試專用的公司代碼

題目需求

Part A：資料表與 Class

- `ZALE09_CTRL`：Key = `mandt` / `bukrs` (來源公司代碼) / `lifnr` (`lifre`，供應商)；欄位 = `bukrs_ar` (`bukrs`，目標公司代碼)、`kunnr` (`kunnr`)、`saknr` (`saknr`)、`blart_ar` (`blart`)、`test_mode` (`xfeld`)、`active` (`xfeld`)
- `ZALE09_QUEUE`：Key = `mandt` / `belnr` (`re_belnr`) / `gjahr` (`gjahr`)；欄位 = `bukrs`、`lifnr` (`lifre`)、`docnum` (`edi_docnum`)、`sent` (`xfeld`)、`errmsg` (`bapi_msg`)、`erdat`、`erzet`
- `ZCL_ALE09_MIRO_OUT`：`enqueue` (靜態，收 RBKP：發票是沖銷單 [`stblg` 非空]、或沒有 `active` 的 `ZALE09_CTRL` 對應時直接離開；否則 `MODIFY` 一筆待送記錄)、`read_doc` (讀 RBKP / `RSEG` 組成內部結構)、`build_idoc` (純邏輯：1 個 `Z1ALE09H` + 每個 `RSEG` 明細 1 個 `Z1ALE09I`；金額用 `{ amount DECIMALS = 2 }` 轉字串)、`send_pending` (掃 `sent` 為空的記錄→組 IDoc→`MASTER_IDOC_DISTRIBUTE`→成功則更新 `docnum` / `sent`、失敗則寫 `errmsg`；不 `commit`，回傳每筆結果)
- `ZCL_ALE09_AR_POST`：`parse`、`build_bapi_data` (純邏輯：依 `ZALE09_CTRL` 與解析結果組出表頭 / AR 行 / GL 行 / 金額行，加總為零)、`process` (依序：解析→找對應設定→冪等檢查→組 BAPI 資料→依 `test_mode` 呼叫 `CHECK` 或 `POST`→判讀 `RETURN` 表)
- `FM Z_IDOC_INPUT_ZALE09` (`ZFG_ALE09`)：比照 ale08 的轉接器結構；訊息可以用 `msgv1` + `msgv2` 分兩段放
- `Report ZR_ALE09_CTRL` (維護控制表；新增前必須驗證目標 / 來源公司代碼、供應商、客戶 [`KNB1`]、收入科目 [`SKB1`] 都真實存在，`test_mode` 預設打開)、`ZR_ALE09_SENDQ` (選取畫面 `p_test`：只列出待送清單；否則送出、成功的 `COMMIT WORK`、列出每筆結果)
- `BAdI Implementation` (GUI：SE19 對 `INVOICE_UPDATE` 建 Implementation，Implementation Class 名稱例如 `ZCL_IM_ALE09_INVUPD`)：`CHANGE_IN_UPDATE` 呼叫 `zcl_ale09_miro_out=>enqueue( s_rbkp_new )`，其他兩個方法留空
- `ABAP Unit`：`build_idoc` (段數、順序、金額字串)、`build_bapi_data` (借貸為零、AR 為正、GL 為負、公司代碼與客戶)、`parse`

Part B：ALE 設定 (GUI，比照 ale07 / ale08)

WE20 Outbound (ZALE09 / ZALE09_BT01)、BD64 加入 ZALE09、WE57 / BD51 / WE42 (Process Code ZALE09→Z_IDOC_INPUT_ZALE09)、WE20 Inbound (ZALE09→ZALE09)。

Part C：端對端

1. ZR_ALE09_CTRL 登記你的測試組合 (test_mode 打開)
2. 對該供應商做一張**真實的 MIRO** (或找一張你有權限產生的測試發票) ——這是真實的會計異動，僅限測試用主資料
3. 存檔後檢查 ZALE09_QUEUE 是否多了一筆 sent 為空的記錄 (BAdI 成功觸發的證據)；用沒登記的供應商做一張發票，確認佇列沒有新記錄 (安全閘生效)
4. 執行 ZR_ALE09_SENDQ，檢查 Outbound IDoc、Inbound IDoc 狀態：預期 Inbound 走到 53，訊息為「檢查通過、未過帳」
5. (選做，明確確認測試公司代碼後) 關閉 test_mode，再對另一張測試發票走一次，確認目標公司代碼出現真實 AR 憑證；完成後用 FB08 沖銷
6. 刻意製造失敗：把 ZALE09_CTRL 的 saknr 改成一個不存在的科目，重送，確認 IDoc 停在 51 並帶著 BAPI 的錯誤訊息；改回正確科目、BD87 重新處理→53

參考答案 (驗證方式)

參考答案程式碼已快照在本目錄：zale09_ctrl.tabl.abap、zale09_queue.tabl.abap、zcl_ale09_miro_out.clas.abap (+.clas.testclasses.abap)、zcl_ale09_ar_post.clas.abap (+.clas.testclasses.abap)、z_idoc_input_zale09.func.abap、zr_ale09_ctrl.prog.abap、zr_ale09_sendq.prog.abap、zcl_im_ale09_invupd.clas.abap (BAdI Implementation 的方法內容)。

⚠ 狀態：草稿，尚未經 SAP 語法檢查與執行驗證 (依賴 Segment 與 SE19 產生的 Implementation Class；BAPI 過帳路徑尚未實測，預設 test_mode 保護)。待驗證清單見 README。

驗證要點：

- SELECT * FROM ZALE09_QUEUE——sent 旗標與 docnum 是否隨排程更新；安全閘測試時是否沒有未登記供應商的記錄
- SELECT DOCNUM, DIRECT, MESTYP, STATUS FROM EDIDC WHERE MESTYP = 'ZALE09' + EDIDS (COUNTR) 歷程
- SELECT SDATA FROM EDID4 WHERE DOCNUM = '<IDoc號>' AND SEGNAME = 'Z1ALE09I'——金額字串格式 (如 100.00)
- 選做過帳時：SELECT BELNR, BLART, BUKRS, XBLNR FROM BKPF WHERE BUKRS = '<目標公司代碼>' AND XBLNR = '<發票號碼+年度>'

思考題

1. 發票被沖銷 (MR8M) 時，enqueue 會直接離開，Paper Company 的 AR 就不會被沖銷。如果要補上「沖銷同步」，你會把它設計成同一個 Message Type 加一個旗標，還是另一個獨立的 Message Type？各自的取舍是什麼？
2. 這題的 AR 只用明細淨額 (不含稅)。如果發票有稅額，Paper Company 那邊的稅該怎麼處理？ (提示：想想 RBKP-RMWW 含稅總額與 RSEG-WRBTR 淨額的差，以及稅額科目在 BAPI_ACC_DOCUMENT_POST 的哪張表)
3. 如果 ZALE09_QUEUE 有一筆記錄反覆送不出去 (例如目標科目一直有誤)，排程報表每輪都會重試它。你會怎麼設計「重試上限」，避免同一筆壞資料永遠占用資源、也不悄悄被忽略？

4. 掛勾用 `CHANGE_IN_UPDATE` 而不是 `CHANGE_AT_SAVE`。如果一個 MIRO 存檔失敗回滾了，兩種時機各自會留下什麼痕跡？

答案

Part A 參考答案見快照檔；Part B 為 GUI 設定；Part C 端對端請回報各步驟結果（佇列狀態、IDoc 號碼與狀態碼、選做的憑證號碼）。⚠ 選做的真實過帳只能在測試專用公司代碼進行。